Pitfalls of Passive Threat Hunting and Tool Reliance

Purple Teaming Series

Image: A Spartan defender facing down a “cozy” bear …

TABLE OF CONTENTS:

Page 3: *Introduction*

Page 4: *Discovering the Specific Flagged Bytes in a Detection with Hades Scanner*

Page 8: *Shellcode Patching*

Page 10: *Modifying/Customizing Open Source Tooling: Havoc PoC*

Page 15: *Key Takeaways and Touching on Behavioral Detections*

Page 16: *Conclusion and References*

Preface/Introduction

Cybersecurity is one of the most dynamic career fields in the world. Thus, innovation fuels the success of both offensive and defensive security postures. The first post in our series will delve into the importance of fostering a culture of innovation, moving beyond compliance, and addressing common pitfalls; mainly the unrealistic perception of the threat landscape and over-reliance on tools.

There are so many risk assessment standards and measures of compliance that companies focus on along with the shiny new tool that promises to reduce risk and stop threats. In this post, we will highlight the importance of the defensive team, their detection logic, and workflows for proactive hunting of threats.

To put it simply, it is inevitable that your tools will fail against an advanced threat. As well as you are able to tune them, reduce false positive expectations, and optimize them for your specific environment they will never be the stand-alone solution for detecting threats. Tools are designed to enable the defender. If the defensive team is not proactively hunting, the effectiveness of the tool is drastically reduced.

We will look at an example of the Havoc Framework, an increasingly popular open source command and control framework, and how it can be configured to bypass detections from Elastic, which are used by their EDR and SIEM solutions.

For the purposes of this write up, we will focus on making simple configuration changes to the source and header files of the demon agent pre-build and shellcode patching to reduce the detection surface for our theoretical implant. The purpose of doing so is to highlight the false sense of security some organizations may feel when assuming their detections based on out of the box configurations are enough to protect them from a determined and highly skilled adversary.

We can demonstrate by using a default out of the box havoc configuration to generate shellcode for our payload. Using a custom tool, we will scan and analyze the raw binary file for detections/alerts. Let’s take a look…

Discovering the Specific Flagged Bytes in a Detection with Hades Scanner
Using 0xtriboulet's phenomenal Hades Scanner tool, we can scan the binary file for detections based on the yara forge repository containing a vast amount of open-source rules and detections used by security teams.

```
\Desktop\implants\Python Maldev>py .\hades_scanner.py --ruleset full .\havoc_x64.bin
```

The results are listed below in italics and specific rules are bolded:

Downloading YARA ruleset: full
Found rule files: ['C:\\Users\\admin\\Desktop\\implants\\Python Maldev\\packages\\full\\yara-rules-full.yar']

Rule: ELASTIC_Windows_Generic_Threat_3F390999
String match: $a1
Offset: 0x00000a30
Data: 104889d9488b5910ff61080f1f40004989cbc34989ca418b430841ff23c39048c1e10431c081e1f00f00004901c84c8d0c024e8d140031c9458a1c0a48

Rule: ELASTIC_Windows_Trojan_Havoc_9C7Bb863
String match: $a1
Offset: 0x00000000
Data: 564889e64883e4f04883ec20e80f0000004889f45ec3
String match: $a2
Offset: 0x000002ea
Data: 65488b042560000000

Rule: ELASTIC_Windows_Trojan_Havoc_88053562
String match: $a
Offset: 0x0000ffcb

Data:
4881ecf8040000488d7c247844898c2458050000488bac24600500004c8d6c2478f3abb95900000
048c744247000000000c744247868000000c78424b4000000

Rule: ELASTIC_Windows_Trojan_Havoc_Ffecc8Af

String match: $commands_table
Offset: 0x000181ff
Data:
0b0000000000000020648199030000006400000000000000003481990300000015000000000000
00704a8199030000001010000000000000305a8199030000000c000000000000000806181990300
00000f0000000000000603d8199030000001400000000000000070498199030000000120000000
0000008030819903000000032000000000000070328199030000000c40900000000000030348199
03000000ce09000000000000a063819903000000d8090000000000000105881990300000034080 0
0000000000905081990300000001600000000000000030468199030000001800000000000000 7047
8199030000001a0000000000000040698199030000000280000000000000000106a819903000000e2
0900000000000005071819903000000ec0900000000000090648199030000000f609000000000000
604b819903000000000a000000000000f04f8199030000005c00000000000000

String match: $hash_ldrloaddll
Offset: 0x0000007f
Data: 436a459e
Offset: 0x00007528
Data: 436a459e
String match: $hash_ntaddbootentry
Offset: 0x00013949
Data: 76c7fc8c
String match: $hash_ntallocatevirtualmemory
Offset: 0x0000008f
Data: ecb883f7
Offset: 0x0000788c
Data: ecb883f7
String match: $hash_ntfreevirtualmemory
Offset: 0x00007b36
Data: 09c60228
String match: $hash_ntunmapviewofsection
Offset: 0x00007b55
Data: cd12a46a
String match: $hash_ntwritevirtualmemory
Offset: 0x00007ad9
Data: 920117c3
String match: $hash_ntsetinformationvirtualmemory

Offset: 0x000077d2
Data: 39c26a94
String match: $hash_ntqueryvirtualmemory
Offset: 0x00007b74
Data: 5de8c010
String match: $hash_ntopenprocesstoken
Offset: 0x00007a5d
Data: 99ca0d35
String match: $hash_ntopenthreadtoken
Offset: 0x000078e9
Data: d2473380

Rule: ELASTIC_Windows_Trojan_Generic_9997489C

String match: $ldrload_dll
Offset: 0x0000007f
Data: 436a459e
Offset: 0x00007528
Data: 436a459e
String match: $loadlibraryw
Offset: 0x00007d10
Data: f12f07b7
String match: $ntallocatevirtualmemory
Offset: 0x0000008f
Data: ecb883f7
Offset: 0x0000788c
Data: ecb883f7
String match: $ntcreatethreadex
Offset: 0x00007a3e
Data: b0cf18af
String match: $ntqueryinformationprocess
Offset: 0x0000784e
Data: c25ddc8c
String match: $ntprotectvirtualmemory
Offset: 0x0000009c
Data: 8828e950
Offset: 0x00007a9b
Data: 8828e950
String match: $ntreadvirtualmemory
Offset: 0x00007b17
Data: 038128a3

String match: $ntwritevirtualmemory
Offset: 0x00007ad9
Data: 920117c3
String match: $rtladdvectoredexceptionhandler
Offset: 0x000076f9
Data: 896cf02d
String match: $rtlallocateheap
Offset: 0x00007547
Data: 5a4ce93b
String match: $rtlqueueworkitem
Offset: 0x0000767d
Data: 8e0292ae
String match: $virtualprotect
Offset: 0x00007d55
Data: 0d5057e8

Rule: EMBEERESEARCH_Win_Havoc_Djb2_Hashing_Routine_Oct_2022

String match: $dll
Offset: 0x0000be9f
Data: b8051500000fbe1148ffc184d274076bc02101d0ebef

Shellcode Patching

As we can see, there are five general rules that were alerted. This seems excellent from a defensive perspective! Let's take the output from the Hades Scanner and examine the disassembly in Binary Ninja.

Rule: ELASTIC_Windows_Generic_Threat_3F390999

String match: $a1

Offset: 0x00000a30

Data:

104889d9488b5910ff61080f1f40004989cbc34989ca418b430841ff23c39048c1e10431c081e1f00f00004901c84c8d0c024e8d140031c9458a1c0a48

```
00000a2d  4883ec10          sub     rsp, 0x10
00000a31  4889d9            mov     rcx, rbx
00000a34  488b5910          mov     rbx, qword [rcx+0x10]
00000a38  ff6108            jmp     qword [rcx+0x8]

00000a3b                                    0f 1f 40 00                ..@.

00000a3f  int64_t sub_a3f() __pure

00000a3f  4989cb            mov     r11, rcx
00000a42  c3                retn     {__return_addr}


00000a43  int64_t sub_a43(void* arg1 @ r11)

00000a43  4989ca            mov     r10, rcx
00000a46  418b4308          mov     eax, dword [r11+0x8]
00000a4a  41ff23            jmp     qword [r11]

00000a4d                                          c3 90                  ..

00000a4f  void* __convention("win64") sub_a4f(int64_t arg1, void* arg2, int64_t arg3)

00000a4f  48c1e104          shl     rcx, 0x4
00000a53  31c0              xor     eax, eax  {0x0}
00000a55  81e1f00f0000      and     ecx, 0xff0
00000a5b  4901c8            add     r8, rcx

00000a5e  4c8d0c02          lea     r9, [rdx+rax]
00000a62  4e8d1400          lea     r10, [rax+r8]
00000a66  31c9              xor     ecx, ecx  {0x0}

00000a68  458a1c0a          mov     r11b, byte [r10+rcx]
00000a6c  48ffc1            inc     rcx
```

If you examine the above screenshot, you can see that I was able to identify the disassembly instructions responsible for the flagged alert. From an attacker's perspective, if there is a single instruction in this sequence that I can change (*patch*) while maintaining similar functionality I can render this detection useless. Because the pattern match detection is such a long string, changing any sequence within the string at any point renders it invalid. Pretty neat! This process is more difficult and annoying with the smaller matches:

Rule: ELASTIC_Windows_Trojan_Generic_9997489C

String match: $ldrload_dll

Offset: 0x0000007f

Data: 436a459e

```
0000007e  ba436a459e          mov     edx, 0x9e456a43
```

As you can see, the above alert match is far smaller and much more annoying to patch.

Let's break down the data that is being flagged/alerted on:

BA is the opcode for the mov instruction with a 32-bit immediate value.
Operand:

43 6A 45 9E is the immediate 32-bit value being moved into the EDX register (0x9e456a43), which is what the alert is flagging on

We can use a tool like defuse to disassemble the value being moved into edx.

Disassembly
Raw Hex (zero bytes in bold):

9E456A43

String Literal:

"\x9E\x45\x6A\x43"

Array Literal:

{ 0x9E, 0x45, 0x6A, 0x43 } Little endian representation of the data alerted on!

Disassembly:
0: 9e sahf
1: 45 6a 43 rex.RB push 0x43

From here an attacker can attempt to modify these instructions in order to bypass this detection. Annoying, but possible.

We now see that shellcode patching is a viable option to reduce the detections of our payload.

Let's examine another method!

Modifying/Customizing Open Source Tooling: Havoc PoC

Havoc is highly customizable by nature, it was not designed to be operated purely out of the box. The author notes several times in the documentation that it is recommended to customize and tailor havoc to your liking and write your own implants as a method of gaining code execution. In other words, it was designed with modularity and customization in the forefront.

Quick Sidebar:

When Havoc is downloaded from its official repository GitHub - HavocFramework/Havoc: The Havoc Framework. , the server and client are not installed as standalone binaries. The user must use make files (an open-source, cross-platform build system that helps manage the build process), to build the binaries. When I first got into offensive security in 2021, Havoc was largely unknown. Havoc's initial release gained traction and was highly touted for bypassing Windows 11 out of the box, with creators like John Hammond making content specifically about the framework (HAVOC C2 - Demon Bypasses Windows 11 Defender). So, naturally, being brand new to the field I was so excited to download the framework and try it myself. I ended up joining the Havoc Discord because I struggled with the simple process of running the pre-configured make files to build the server and client binaries. At the time I found this frustrating because I did not understand the benefit of being able to customize the code of the framework itself.

This example highlights the benefits of a customizable/modular tool like Havoc. Referencing the [Havoc Documentation](#) , once you pull the repository from GitHub you simply pull dependencies and build the server/client binaries with 'make'. However, before building the binary, we can change the source code to avoid detections.

For our example, let's reference the output of our Hades Scanner for the out of the box demon agent shellcode:

Rule: EMBEERESEARCH_Win_Havoc_Djb2_Hashing_Routine_Oct_2022

String match: $dll
Offset: 0x0000be9f
Data: b8051500000fbe1148ffc184d274076bc02101d0ebef

```
0000be9f  int64_t __convention("win64") sub_be9f(char* arg1)

0000be9f  b805150000        mov     eax, 0x1505

0000bea4  0fbe11            movsx   edx, byte [rcx]
0000bea7  48ffc1            inc     rcx
0000beaa  84d2              test    dl, dl
0000beac  7407              je      0xbeb5

0000beae  6bc021            imul    eax, eax, 0x21
0000beb1  01d0              add     eax, edx
0000beb3  ebef              jmp     0xbea4

0000beb5  c3                retn     {__return_addr}
```

Notice the alert references a hashing routine "Djb2". With a little bit of digging in the Demon agent source, there are references to this hashing method in the form of a python script, a source file, and a header file. By changing a few values and making them consistent across these files before building, we are able to effectively bypass this alert in our shellcode.

In essence,

1. Make modifications to the necessary files referencing the hashing routine
2. Rebuild the server/client binaries
3. Generate the shellcode through havoc with the same parameters as before
4. Test our shellcode with Hades Scanner to see if we were able to eliminate this detection

Examine the following results of scanning the shellcode generated by the rebuilt havoc:

```
C:\Users\[redacted]\Desktop\implants\Python Maldev>py .\hades_scanner_hex_offsets.py --ruleset full .\havoc_rebuild_x64.bin
Downloading YARA ruleset: full
Found rule files: ['C:\\Users\\[redacted]\Desktop\\implants\\Python Maldev\\packages\\full\\yara-rules-full.yar']
Rule: ELASTIC_Windows_Generic_Threat_3F390999
        String match: $a1
        Offset: 0x00000a30
        Data: 104889d9488b5910ff61080f1f40004989cbc34989ca418b430841ff23c39048c1e10431c081e1f00f00004901c84c8d0c024e8d140031c9458a1c0a48
Rule: ELASTIC_Windows_Trojan_Havoc_9C7Bb863
        String match: $a1
        Offset: 0x00000000
        Data: 564889e64883e4f04883ec20e80f0000004889f45ec3
        String match: $a2
        Offset: 0x000002ea
        Data: 65488b042560000000
Rule: ELASTIC_Windows_Trojan_Havoc_88053562
        String match: $a
        Offset: 0x0000ffcb
        Data: 4881ecf8040000488d7c247844898c2458050000488bac24600500004c8d6c2478f3abb95900000048c744247000000000c744247868000000c78424b4000000
Rule: ELASTIC_Windows_Trojan_Havoc_Ffecc8Af
        String match: $commands_table
        Offset: 0x000181ff
        Data: 0b0000000000000000020641c8302000000640000000000000000341c8302000000150000000000000704a1c8302000000101000000000000305a1c83020000000c000000000000000080611c83020000000f0000000000
001200000000000000080301c8302000000032000000000000070321c8302000000c4090000000000000030341c8302000000ce0900000000000a0631c8302000000d8090000000000000010581c8302000000340800000000000000090501c830
000070471c8302000000001a0000000000000000040691c8302000000280000000000000000106a1c8302000000e2090000000000000050711c8302000000ec0900000000000000090641c8302000000f6090000000000000000604b1c83020000000000a00
        String match: $hash_ldrloaddll
        Offset: 0x0000007f
        Data: 436a459e
        Offset: 0x00007528
        Data: 436a459e
        String match: $hash_ntaddbootentry
        Offset: 0x00013949
        Data: 76c7fc8c
        String match: $hash_ntallocatevirtualmemory
        Offset: 0x0000008f
        Data: ecb883f7
        Offset: 0x0000788c
        Data: ecb883f7
        String match: $hash_ntfreevirtualmemory
        Offset: 0x00007b36
        Data: 09c60228
        String match: $hash_ntunmapviewofsection
        Offset: 0x00007b55
        Data: cd12a46a
        String match: $hash_ntwritevirtualmemory
        Offset: 0x00007ad9
        Data: 920117c3
        String match: $hash_ntsetinformationvirtualmemory
        Offset: 0x000077d2
        Data: 39c26a94
        String match: $hash_ntqueryvirtualmemory
        Offset: 0x00007b74
        Data: 5de8c010
        String match: $hash_ntopenprocesstoken
        Offset: 0x00007a5d
        Data: 99ca0d35
        String match: $hash_ntopenthreadtoken
        Offset: 0x000078e9
        Data: d2473380
Rule: ELASTIC_Windows_Trojan_Generic_9997489C
        String match: $ldrload_dll
        Offset: 0x0000007f
```

```
        Data: 99ca0d35
        String match: $hash_ntopenthreadtoken
        Offset: 0x000078e9
        Data: d2473380
Rule: ELASTIC_Windows_Trojan_Generic_9997489C
        String match: $ldrload_dll
        Offset: 0x0000007f
        Data: 436a459e
        Offset: 0x00007528
        Data: 436a459e
        String match: $loadlibraryw
        Offset: 0x00007d10
        Data: f12f07b7
        String match: $ntallocatevirtualmemory
        Offset: 0x0000008f
        Data: ecb883f7
        Offset: 0x0000788c
        Data: ecb883f7
        String match: $ntcreatethreadex
        Offset: 0x00007a3e
        Data: b0cf18af
        String match: $ntqueryinformationprocess
        Offset: 0x0000784e
        Data: c25ddc8c
        String match: $ntprotectvirtualmemory
        Offset: 0x0000009c
        Data: 8828e950
        Offset: 0x00007a9b
        Data: 8828e950
        String match: $ntreadvirtualmemory
        Offset: 0x00007b17
        Data: 038128a3
        String match: $ntwritevirtualmemory
        Offset: 0x00007ad9
        Data: 920117c3
        String match: $rtladdvectoredexceptionhandler
        Offset: 0x000076f9
        Data: 896cf02d
        String match: $rtlallocateheap
        Offset: 0x00007547
        Data: 5a4ce93b
        String match: $rtlqueueworkitem
        Offset: 0x0000767d
        Data: 8e0292ae
        String match: $virtualprotect
        Offset: 0x00007d55
        Data: 0d5057e8

C:\Users\[redacted]\Desktop\implants\Python Maldev>_
```

Notice that we were able to effectively eliminate the detection for the hashing routine!

Let's examine the disassembly in BinaryNinja:

Alerted code from out-of-the-box shellcode:

```
0000be9f  int64_t __convention("win64") sub_be9f(char* arg1)

0000be9f  b805150000        mov     eax, 0x1505

0000bea4  0fbe11            movsx   edx, byte [rcx]
0000bea7  48ffc1            inc     rcx
0000beaa  84d2              test    dl, dl
0000beac  7407              je      0xbeb5

0000beae  6bc021            imul    eax, eax, 0x21
0000beb1  01d0              add     eax, edx
0000beb3  ebef              jmp     0xbea4

0000beb5  c3                retn     {__return_addr}
```

Shellcode from customized Havoc build:

```
0000be9f  int64_t __convention("win64") sub_be9f(char* arg1)

0000be9f  b806150000        mov     eax, 0x1506

0000bea4  0fbe11            movsx   edx, byte [rcx]
0000bea7  48ffc1            inc     rcx
0000beaa  84d2              test    dl, dl
0000beac  7407              je      0xbeb5

0000beae  6bc021            imul    eax, eax, 0x21
0000beb1  01d0              add     eax, edx
0000beb3  ebef              jmp     0xbea4

0000beb5  c3                retn     {__return_addr}
```

Can you spot the difference? It is a singular bit difference between the two as a result of changing the values in the source and header files by one-off! Examine the value being moved into the accumulator register (eax) at offset 0000be9f. This further illustrates the point made earlier in the post, that changing a single value in a string being alerted on negates the validity of the entire detection.

Flagged:b8051500000fbe1148ffc184d274076bc02101d0ebef
Evaded:b8061500000fbe1148ffc184d274076bc02101d0ebef

Key Takeaways

The main purpose of this article is twofold. One, to highlight that a persistent and determined threat will find ways to overcome your toolset. Two, because of this, the proactive defender and manual threat hunting is often the difference between preventing a breach and dealing with damage control from a successful one. The big takeaway: never assume the threat landscape consists exclusively of out-of-the-box open source tooling. Never underestimate the enemy and assume they are able to evade your defenses. If they do, what workflows and procedures can you use to systematically hunt, clear, and coordinate with proper authorities for response?

For more expert level guidance on how to improve your detection logic, workflows, and overall capabilities/efficiency of your blue team you can reference this article as a good start.

To recap, we covered two different ways attackers may be able to minimize detections on payloads from open source frameworks: shellcode patching and source code customization. We also highlighted the downfall of static or signature based detections often used by security solutions to proactively kill malicious processes.

A Note on Behavioral Detections…

Behavioral detections are still a beast of their own. Though they are incredibly useful and far more difficult to evade, attackers are still able to analyze how specific solutions perform their analysis and develop techniques to evade them. Elastic's EDR, for example, is currently leading the way in stack tracing/call stack analysis. Each entry in a stack trace represents a function call, showing the function name and often the file and line number where the function was called. Additionally a program counter indicates the exact location in the code where the error or exception occurred. The call sequence or order of function calls, starting from the most recent call (the top of the stack) and going back to the initial function call (the bottom of the stack) are also important analytical pieces for EDR using call stack analysis.

As awesome as this is, attackers can overwrite the memory regions containing the original shellcode with NOPs or other non-functional instructions. This obfuscation makes it challenging to determine what the original instructions were, hindering the ability to trace the exact execution path. Techniques like inline assembly can be used to directly resolve useful libraries and functions to avoid using common implementations that may be flagged as suspicious. To detonate or execute the shellcode, rather than using a function pointer or creating a new thread, attackers can use genuine and uncommon function calls that allow for callback function execution. Sleep obfuscation, IAT hooking, and several other methods of bypassing behavior detections always exist as well.

Conclusion
All of this to highlight once again that the tools will fail to always give the proper perspective to a defender. It is absolutely critical to understand the organizational threat landscape, baseline activity, and the mechanisms of the tooling being used.

In conclusion, I hope you found this beneficial if you read this far! If you have any questions about the content, feel free to reach out!

//This write up was developed by Steven Adams:
Patreon

A huge thanks once again to 0xtriboulet for the tooling and fundamental workflow. I highly recommend following his work!

References:

Technical Reading Material
RedTeam.Guide
Capability Abstraction
Detection Spectrum. Have you ever heard someone call… | by Jared Atkinson
Introducing the Funnel of Fidelity | by Jared Atkinson

Tools:
Online x86 / x64 Assembler and Disassembler
Binary Ninja
Hades Scanner v0.2 | Patreon
GitHub - HavocFramework/Havoc: The Havoc Framework.

Communities:
Havoc Discord